

Adaptive RAG – Summary

This notebook implements an **Adaptive Retrieval-Augmented Generation (RAG)** system using LangChain and LLMs.

Unlike traditional RAG, this system dynamically decides whether to retrieve information from:

- Vector Database (Chroma)
- Web Search

Key Components

- HuggingFace Embeddings (all-MiniLM-L6-v2)
- Chroma Vector Store
- LLM-based Query Router
- Document Relevance Grader (Yes/No filtering)
- GROQ LLM (openai/gpt-oss-120b)

Workflow

User Question → Router → Retrieve (VectorDB/Web) → Relevance Check → Final Response

Benefit

- Better accuracy
- Reduced hallucinations
- Intelligent source selection
- More production-ready RAG architecture

```
from langchain_text_splitters.character import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import Chroma
from langchain_groq import ChatGroq
import os
from langchain_huggingface import HuggingFaceEmbeddings
```

load environment variables in Python

```
import os
from dotenv import load_dotenv
load_dotenv()
GOOGLE_API_KEY=os.getenv("GOOGLE_API_KEY")
TAVILY_API_KEY=os.getenv("TAVILY_API_KEY")
GROQ_API_KEY=os.getenv("GROQ_API_KEY")
LANGCHAIN_API_KEY=os.getenv("LANGCHAIN_API_KEY")
os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
os.environ["TAVILY_API_KEY"] = TAVILY_API_KEY
```

```
os.environ["GROQ_API_KEY"] = GROQ_API_KEY
os.environ["LANGCHAIN_API_KEY"] = LANGCHAIN_API_KEY
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_ENDPOINT"] = "https://api.smith.langchain.com"
```

Chat Model LLM using GROQ API

```
llm=ChatGroq(model_name="openai/gpt-oss-120b")

llm.invoke("hi").content

'Hello! How can I help you today?'
```

Load Embedding Model

```
embeddings=HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

Load Data

```
urls = [
    "https://lilianweng.github.io/posts/2023-06-23-agent/",
    "https://lilianweng.github.io/posts/2023-03-15-prompt-
engineering/",
    "https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/",
]

docs = [WebBaseLoader(url).load() for url in urls]
docs_list = [item for sublist in docs for item in sublist]
```

setup TextSplitter

```
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=500, chunk_overlap=0
)
doc_splits = text_splitter.split_documents(docs_list)
```

Create VectorStore

```
vectorstore = Chroma.from_documents(
    documents=doc_splits,
    collection_name="rag-chroma",
    embedding=embeddings,
)
```

Create retriever

```
retriever = vectorstore.as_retriever()
```

```

from typing import Literal

from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field

```

Query Routing with Structured LLMs

```

# Data model
class RouteQuery(BaseModel):
    """Route a user query to the most relevant datasource."""

    # Decide which data source to use
    datasource: Literal["vectorstore", "web search"] = Field(
        ...,
        # Instruction for the LLM on how to choose the datasource
        description="Given a user question choose to route it to web search or a vectorstore.",
    )

# Wrap LLM to return structured output based on RouteQuery schema
structured_llm_router = llm.with_structured_output(RouteQuery)

# System prompt defining routing rules
system = """You are an expert at routing a user question to a vectorstore or web search.
The vectorstore contains documents related to agents, prompt engineering, and adversarial attacks.
Use the vectorstore for questions on these topics. Otherwise, use web-search."""

# Prompt template combining system and user input
route_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),      # System-level instructions
        ("human", "{question}"), # User question placeholder
    ]
)

# Chain prompt with structured LLM to create router
question_router = route_prompt | structured_llm_router

print(
    question_router.invoke(
        {"question": "Who will the Bears draft first in the NFL draft?"}
    )
)

datasource='web search'

```

```
print(question_router.invoke({"question": "What are the types of agent memory?"}))

datasource='vectorstore'
```

Retrieval Relevance Grader

```
### Retrieval Grader

# Data model for grading retrieved documents
class GradeDocuments(BaseModel):
    """Binary score for relevance check on retrieved documents."""

    # Relevance decision: yes or no
    binary_score: str = Field(
        # Instruction for LLM to output relevance label
        description="Documents are relevant to the question, 'yes' or 'no'"
    )

# Configure LLM to return structured grading output
structured_llm_grader = llm.with_structured_output(GradeDocuments)

# System prompt defining grading criteria
system = """You are a grader assessing relevance of a retrieved document to a user question.
    If the document contains keyword(s) or semantic meaning related to the user question, grade it as relevant.
    It does not need to be a stringent test. The goal is to filter out erroneous retrievals.
    Give a binary score 'yes' or 'no' score to indicate whether the document is relevant to the question."""

# Prompt template combining document and question
grade_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system), # Grading rules and instructions
        ("human", "Retrieved document: \n\n {document} \n\n User question: {question}"),
    ]
)

# Chain prompt with structured LLM to create retrieval grader
retrieval_grader = grade_prompt | structured_llm_grader

question = "agent memory"

docs = retriever.invoke(question)

doc_txt = docs[1].page_content
```

```
print(retrieval_grader.invoke({"question": question, "document":
doc_txt}))

binary_score='no'
```

Generating Answers with RAG Chain

```
# Generating Answers with RAG Chain
### Generate

from langchain_classic import hub
from langchain_core.output_parsers import StrOutputParser

# Pull a pre-built RAG prompt from LangChain Hub
prompt = hub.pull("rlm/rag-prompt")

# Function to format retrieved documents into a single string
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

# Chain: prompt -> LLM -> string output parser
rag_chain = prompt | llm | StrOutputParser()

# Run
generation = rag_chain.invoke({"context": docs, "question": question})
print(generation)
```

In LLM-powered agents, short-term memory refers to the in-context learning the model uses during a single interaction, while long-term memory is an external store (often a vector database) that lets the agent retain and retrieve information across sessions. This combination lets the agent reason with recent context and also recall persistent knowledge over time.

Grading LLM Generations for Hallucinations

```
### Hallucination Grader

# Data model for grading hallucinations in LLM outputs
class GradeHallucinations(BaseModel):
    """Binary score for hallucination present in generated answer."""

    # Binary decision: is the generation grounded in facts?
    binary_score: str = Field(
        # Instruction for LLM to output 'yes' or 'no'
        description="Answer is grounded in the facts, 'yes' or 'no'"
    )

# Configure LLM to return structured grading output
structured_llm_grader =
llm.with_structured_output(GradeHallucinations)
```

```

# System prompt defining hallucination grading criteria
system = """You are a grader assessing whether an LLM generation is
grounded in / supported by a set of retrieved facts.
Give a binary score 'yes' or 'no'. 'Yes' means that the answer is
grounded in / supported by the set of facts."""

# Prompt template combining retrieved documents and LLM generation
hallucination_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", "Set of facts: \n\n {documents} \n\n LLM generation:
{generation}"),
    ]
)

# Chain prompt with structured LLM to create hallucination grader
hallucination_grader = hallucination_prompt | structured_llm_grader

hallucination_grader.invoke({"documents": docs, "generation":
generation})

GradeHallucinations(binary_score='yes')

```

Rewriting Questions for Vectorstore Retrieval

```

### Question Re-writer

# System prompt defining the role of the question re-writer
system = """You are a question re-writer that converts an input
question to a better version that is optimized
for vectorstore retrieval. Look at the input and try to reason
about the underlying semantic intent / meaning."""

# Create a prompt template combining system instructions and user
question
re_write_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system), # Instructions for the rewriter
        (
            "human",
            "Here is the initial question: \n\n {question} \n
Formulate an improved question.", # Placeholder for input
        ),
    ]
)

# Chain: prompt -> LLM -> string output parser
question_rewriter = re_write_prompt | llm | StrOutputParser()

```

```

# Invoke the rewriter with a user question
question_rewriter.invoke({"question": question})

'''Improved question:** \n*What is “agent memory” in the context of
AI systems, and how is it implemented and utilized to enable an agent
to retain and recall information across interactions?*'

### Search

from langchain_community.tools.tavily_search import
TavilySearchResults

web_search_tool = TavilySearchResults(k=3)

C:\Users\Suresh Beekhiani\AppData\Local\Temp\
ipykernel_14720\730760015.py:5: LangChainDeprecationWarning: The class
`TavilySearchResults` was deprecated in LangChain 0.3.25 and will be
removed in 1.0. An updated version of the class exists in the
`langchain-tavily` package and should be used instead. To use it run
`pip install -U `langchain-tavily` and import as `from
`langchain_tavily import TavilySearch``.
web_search_tool = TavilySearchResults(k=3)

```

Graph State Schema

```

from typing import List
from typing_extensions import TypedDict

# Define the schema for the state stored in the graph
class GraphState(TypedDict):
    """
    Represents the state of our graph.

    Attributes:
        question: The user's input question
        generation: The LLM-generated answer
        documents: List of retrieved documents used for generation
    """

    # User input question
    question: str

    # Generated answer from LLM
    generation: str

    # List of retrieved documents relevant to the question
    documents: List[str]

```

End-to-End RAG Workflow with Document Grading and Question Routing

```
from langchain_core.documents import Document

### Node Functions for Graph Workflow ###

def retrieve(state):
    """
    Retrieve documents based on the user's question.

    Args:
        state (dict): The current graph state

    Returns:
        dict: Updates state with 'documents' key containing retrieved documents
    """
    print("---RETRIEVE---")
    question = state["question"]

    # Invoke retriever to get documents relevant to the question
    documents = retriever.invoke(question)
    return {"documents": documents, "question": question}

def generate(state):
    """
    Generate an answer using RAG chain from retrieved documents.

    Args:
        state (dict): The current graph state

    Returns:
        dict: Updates state with 'generation' key containing LLM output
    """
    print("---GENERATE---")
    question = state["question"]
    documents = state["documents"]

    # Use RAG chain to generate answer from context documents
    generation = rag_chain.invoke({"context": documents, "question": question})
    return {"documents": documents, "question": question, "generation": generation}

def grade_documents(state):
    """
    Check if retrieved documents are relevant to the question.
```

```

    Args:
        state (dict): The current graph state

    Returns:
        dict: Updates 'documents' with only relevant documents
    """
    print("---CHECK DOCUMENT RELEVANCE TO QUESTION---")
    question = state["question"]
    documents = state["documents"]

    # Filter documents using the retrieval grader
    filtered_docs = []
    for d in documents:
        score = retrieval_grader.invoke({"question": question,
                                         "document": d.page_content})
        grade = score.binary_score
        if grade == "yes":
            print("---GRADE: DOCUMENT RELEVANT---")
            filtered_docs.append(d)
        else:
            print("---GRADE: DOCUMENT NOT RELEVANT---")
    return {"documents": filtered_docs, "question": question}

def transform_query(state):
    """
    Re-write the user's question to improve retrieval.

    Args:
        state (dict): The current graph state

    Returns:
        dict: Updates 'question' with re-written query
    """
    print("---TRANSFORM QUERY---")
    question = state["question"]
    documents = state["documents"]

    # Use question re-writer to optimize query for retrieval
    better_question = question_rewriter.invoke({"question": question})
    return {"documents": documents, "question": better_question}

def web_search(state):
    """
    Perform a web search based on the re-written question.

    Args:
        state (dict): The current graph state

```

```

Returns:
    dict: Updates 'documents' key with retrieved web results
    """
print("---WEB SEARCH---")
question = state["question"]

# Call web search tool
docs = web_search_tool.invoke({"query": question})
web_results = "\n".join([d["content"] for d in docs])
web_results = Document(page_content=web_results)

return {"documents": web_results, "question": question}

### Edge / Decision Functions ###

def route_question(state):
    """
    Route question to web search or vectorstore (RAG) based on LLM
    routing.

    Args:
        state (dict): The current graph state

    Returns:
        str: Next node to call in the workflow
    """
print("---ROUTE QUESTION---")
question = state["question"]

# Use structured LLM router to decide datasource
source = question_router.invoke({"question": question})
if source.datasource == "web_search":
    print("---ROUTE QUESTION TO WEB SEARCH---")
    return "web_search"
elif source.datasource == "vectorstore":
    print("---ROUTE QUESTION TO RAG---")
    return "vectorstore"

def decide_to_generate(state):
    """
    Decide whether to generate an answer or re-write the question.

    Args:
        state (dict): The current graph state

    Returns:
        str: Next node to call

```

```

"""
print("---ASSESS GRADED DOCUMENTS---")
filtered_documents = state["documents"]

if not filtered_documents:
    # No relevant documents, re-generate query
    print("---DECISION: ALL DOCUMENTS ARE NOT RELEVANT, TRANSFORM
QUERY---")
    return "transform_query"
else:
    # Relevant documents exist, proceed to generation
    print("---DECISION: GENERATE---")
    return "generate"

def grade_generation_v_documents_and_question(state):
    """
    Check if LLM generation is grounded in documents and answers the
    question.

    Args:
        state (dict): The current graph state

    Returns:
        str: Decision for next node to call
    """
    print("---CHECK HALLUCINATIONS---")
    question = state["question"]
    documents = state["documents"]
    generation = state["generation"]

    # Grade whether generation is grounded in documents
    score = hallucination_grader.invoke({"documents": documents,
"generation": generation})
    grade = score.binary_score

    if grade == "yes":
        print("---DECISION: GENERATION IS GROUNDED IN DOCUMENTS---")
        # Check if generation answers the question
        print("---GRADE GENERATION vs QUESTION---")
        score = answer_grader.invoke({"question": question,
"generation": generation})
        grade = score.binary_score
        if grade == "yes":
            print("---DECISION: GENERATION ADDRESSES QUESTION---")
            return "useful"
        else:
            print("---DECISION: GENERATION DOES NOT ADDRESS
QUESTION---")
            return "not useful"

```

```

else:
    print("---DECISION: GENERATION IS NOT GROUNDED, RE-TRY---")
    return "not supported"

```

End-to-End RAG Workflow in LangGraph

```

from langgraph.graph import END, StateGraph, START

# Initialize a stateful workflow using the GraphState schema
workflow = StateGraph(GraphState)

# -----
# Define workflow nodes
# -----
workflow.add_node("web_search", web_search)           # Node for
performing web search
workflow.add_node("retrieve", retrieve)               # Node for
retrieving documents from vectorstore
workflow.add_node("grade_documents", grade_documents) # Node for
grading relevance of retrieved documents
workflow.add_node("generate", generate)               # Node for
generating answer from RAG chain
workflow.add_node("transform_query", transform_query) # Node for
rewriting query if needed

# -----
# Build the graph edges
# -----

# Start node: decide routing based on LLM question router
workflow.add_conditional_edges(
    START,                                           # Starting point
    route_question,                                 # Function that decides routing
    {
        "web_search": "web_search",               # Route to web search
        "vectorstore": "retrieve",                 # Route to RAG retrieval
    },
)

# Connect web_search to generate answer
workflow.add_edge("web_search", "generate")

# Connect retrieve to document grading
workflow.add_edge("retrieve", "grade_documents")

# Conditional edges after grading documents
workflow.add_conditional_edges(
    "grade_documents",
    decide_to_generate,                             # Decide next step based on graded
docs

```

```

    {
        "transform_query": "transform_query", # No relevant docs →
        "generate": "generate",              # Relevant docs →
    },
)

# After transforming query, route back to retrieval
workflow.add_edge("transform_query", "retrieve")

# Conditional edges after generation: check if generation is grounded
# and useful
workflow.add_conditional_edges(
    "generate",
    grade_generation_v_documents_and_question, # Function to grade
    {
        "not supported": "generate",          # Generation not grounded →
        "useful": END,                        # Generation is good → end
    },
)

<langgraph.graph.state.StateGraph at 0x261bb93c440>

```

Compile Workflow

```

# Compile the workflow into an executable LangGraph app
app = workflow.compile()

from pprint import pprint

# Run
inputs = {
    "question": "What player at the Bears expected to draft first in
the 2024 NFL draft?"
}
for output in app.stream(inputs):
    for key, value in output.items():
        # Node
        pprint(f"Node '{key}':")
        # Optional: print full state at each node
        # pprint.pprint(value["keys"], indent=2, width=80, depth=None)
    pprint("\n---\n")

```

```

# Final generation
pprint(value["generation"])

---ROUTE QUESTION---
---ROUTE QUESTION TO WEB SEARCH---
---WEB SEARCH---
"Node 'web_search':"
'\n---\n'
---GENERATE---
---CHECK HALLUCINATIONS---
---DECISION: GENERATION IS GROUNDED IN DOCUMENTS---
---GRADE GENERATION vs QUESTION---
---DECISION: GENERATION ADDRESSES QUESTION---
"Node 'generate':"
'\n---\n'
('The Chicago Bears are expected to use the No.\u202f1 overall pick in
the '
'2024 NFL Draft on USC quarterback **Caleb\u202fWilliams**. This
expectation '
'solidified after they traded away Justin\u202fFields. Williams is
widely '
'projected as the Bears' first-round selection.')

# Run
inputs = {"question": "What are the types of agent memory?"}
for output in app.stream(inputs):
    for key, value in output.items():
        # Node
        pprint(f"Node '{key}':" )
        # Optional: print full state at each node
        # pprint.pprint(value["keys"], indent=2, width=80, depth=None)
    pprint("\n---\n")

# Final generation
pprint(value["generation"])

---ROUTE QUESTION---
---ROUTE QUESTION TO RAG---
---RETRIEVE---
"Node 'retrieve':"
'\n---\n'
---CHECK DOCUMENT RELEVANCE TO QUESTION---
---GRADE: DOCUMENT RELEVANT---
---GRADE: DOCUMENT NOT RELEVANT---
---GRADE: DOCUMENT RELEVANT---
---GRADE: DOCUMENT RELEVANT---
---ASSESS GRADED DOCUMENTS---
---DECISION: GENERATE---
"Node 'grade_documents':"
'\n---\n'

```

```
---GENERATE---
---CHECK HALLUCINATIONS---
---DECISION: GENERATION IS GROUNDED IN DOCUMENTS---
---GRADE GENERATION vs QUESTION---
---DECISION: GENERATION ADDRESSES QUESTION---
"Node 'generate':"
'\n---\n'
('Agent memory is typically divided into two categories: **short-term
'
'memory**, which uses the model's in-context context to retain recent
'
'information, and **long-term memory**, which stores knowledge
persistently '
'via external stores such as vector databases. Short-term memory
handles '
'immediate reasoning, while long-term memory enables recall of
information '
'over extended periods. Together they let an LLM-powered agent both
remember '
'the current conversation and retrieve past experience when needed.')
```